

TCP Server Experiment

TCP Server Experiment

- 1. TCP Overview
- 2. Core Concepts
 - 2.1 TCP Three-Way Handshake
 - 2.2 TCP vs UDP API Comparison
 - 2.3 Three Return Values of `recv()`
- 3. Project Architecture and Flow
 - 3.1 TCP Server
- 4. Hardware Dependencies
- 5. Source Code Analysis — TCP Client
 - 5.3 `vTcpClientTask()` — TCP Client Core
- 5. Source Code Analysis — TCP Server
 - 5.1 Macro Configuration
 - 5.2 WiFi Initialization (Review)
 - 5.3 `vTcpServerTask()` — TCP Server Core
- 6. Operation Flow
 - 6.1 Build and Flash Flow
 - 6.2 Testing Method
 - 6.3 Normal Log — TCP Server
- 7. Common Problems

1. TCP Overview

TCP (Transmission Control Protocol) is a **connection-oriented, reliable, byte-stream** transport layer protocol. It establishes connections via three-way handshake, ensuring data arrives in order and intact.

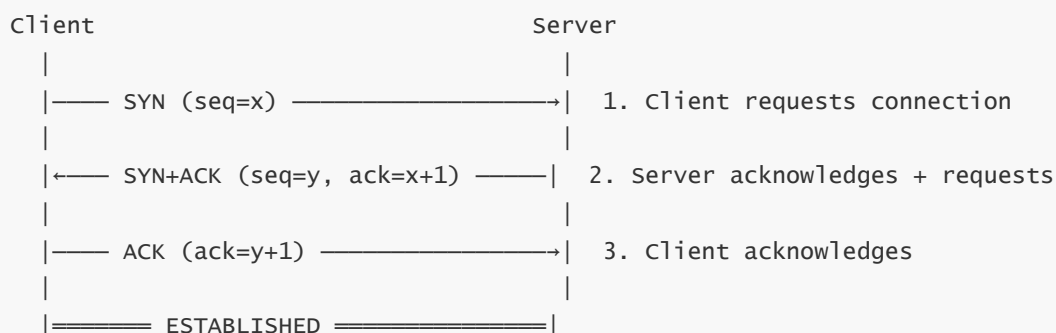
Feature	TCP	UDP
Connection	Three-way handshake	Connectionless
Reliability	Guaranteed delivery, guaranteed order	Not guaranteed
Flow/ Congestion Control	Built-in sliding window, slow start	None
Transmission Mode	Byte stream (no message boundaries)	Datagram (has message boundaries)
Server API	<code>bind</code> → <code>listen</code> → <code>accept</code> → <code>recv/send</code>	<code>bind</code> → <code>recvfrom/sendto</code>
Client API	<code>socket</code> → <code>connect</code> → <code>recv/send</code>	<code>socket</code> → <code>sendto/recvfrom</code>

TCP is the foundation of IoT communications. Whether HTTP web requests, MQTT message publishing, OTA firmware upgrades, or database remote connections, all rely on TCP's reliable transmission at the bottom. In industrial automation, smart home, and remote monitoring scenarios, devices report sensor data and receive control instructions—all requiring "no data loss, no order disruption"—which is TCP's core value. UDP is fast but unreliable; TCP has the overhead of three-way handshake but gains an **end-to-end acknowledgment and retransmission mechanism**, irreplaceable in most IoT scenarios.

This project contains both TCP client and server routines: the client connects to the server and sends data periodically; the server listens on a port, accepts client connections, sends/receives data, and supports reconnecting after client disconnection.

2. Core Concepts

2.1 TCP Three-Way Handshake



In code, the client calls `connect()` to trigger the handshake, and the server calls `accept()` to wait for handshake completion.

2.2 TCP vs UDP API Comparison

```
// === TCP Server ===
socket(AF_INET, SOCK_STREAM, 0)
bind(fd, addr, len)
listen(fd, 5)
client_fd = accept(fd, ...)
recv(client_fd, buf, len, 0)
send(client_fd, buf, len, 0)
```

```
// === TCP Client ===
socket(AF_INET, SOCK_STREAM, 0)
connect(fd, &addr, len)
send(fd, buf, len, 0)
recv(fd, buf, len, 0)
```

```
// === UDP Server ===
socket(AF_INET, SOCK_DGRAM, 0)
bind(fd, addr, len)
// No listen
// No accept, use recvfrom directly
recvfrom(fd, buf, len, 0, &addr, &len)
sendto(fd, buf, len, 0, &addr, len)
```

```
// === UDP Client ===
socket(AF_INET, SOCK_DGRAM, 0)
// No connect
sendto(fd, buf, len, 0, &addr, len)
recvfrom(fd, buf, len, 0, &addr, &len)
```

2.3 Three Return Values of `recv()`

Return Value	Meaning	Handling
<code>> 0</code>	Received n bytes of data	Process normally
<code>= 0</code>	Peer closed connection normally (sent FIN)	<code>close()</code> and <code>break</code>
<code>< 0</code>	No data available (non-blocking) or error	Ignore with <code>MSG_DONTWAIT</code> ; check <code>errno</code> for others

`recv() == 0` is a TCP-specific "peer closed" signal; UDP has no such concept.

3. Project Architecture and Flow

3.1 TCP Server

```
app_main()
├─ wifi_init()           ← WiFi STA init (wait for IP)
├─
├─ xTaskCreate(vTcpServerTask, ...) ← Create TCP server task
├─ vTcpServerTask():
│   ├─ socket(AF_INET, SOCK_STREAM, 0) → Create TCP socket
│   ├─ bind(INADDR_ANY, 8080)          → Bind port
│   ├─ listen(fd, 5)                   → Start listening (max 5 queued)
│   └─ while(1):
│       ├─ accept() [blocking] → wait for client connection
│       └─ inner while(1):
│           ├─ recv(MSG_DONTWAIT) → Non-blocking receive
│           ├─ data received → send() reply
│           ├─ recv==0 → client disconnected, break
│           └─ close(client_fd) → Return to outer accept
```

4. Hardware Dependencies

- **Hardware Connection:** No external wiring, WiFi wireless communication
- **Required Components:** `lwip/sockets`, `esp_wifi`, `esp_netif`, `esp_event`, `nvs_flash`, `FreeRTOS`

5. Source Code Analysis — TCP Client

Full source: `Source Code` → `ESP-IDF` → `5.Network_Protocol` → `4.TCP` → `TCP_Server`

5.3 vTcpClientTask() — TCP Client Core

The biggest difference from UDP client: TCP is "make a call before speaking"—`connect()` three-way handshake succeeds before `send()` / `recv()`. UDP is "send a letter directly"—`sendto()` doesn't need a connection first. Also TCP uses `SOCK_STREAM` (streaming), UDP uses `SOCK_DGRAM` (datagram).

In the send-receive loop, a key judgment—`recv()` returning 0 means the server normally closed the connection (sent FIN package), then `break` out to close socket and delete task. UDP has no this problem because UDP is connectionless, `recvfrom()` never returns 0.

5. Source Code Analysis — TCP Server

Full source: `Source Code` → `ESP-IDF` → `5.Network_Protocol` → `4.TCP` → `TCP_Server`

5.1 Macro Configuration

These macros define WiFi connection info and TCP server address. Modify SSID, password, and target IP according to your network environment.

```
#define WIFI_SSID      "Your_WiFi_SSID"      // WiFi SSID
#define WIFI_PASS      "Your_WiFi_Password"  // WiFi password
#define TCP_SERVER_IP  "192.168.11.183"      // Server IP
#define TCP_SERVER_PORT 8080                 // Server port
#define WIFI_TIMEOUT_MS 15000
```

5.2 WiFi Initialization (Review)

The WiFi STA connection code in this experiment (`wifi_init_sta()`, event callback, NVS init, etc.) is identical to **5.2 WIFI-STA Mode**. `app_main()` does NVS init → WiFi connection to get IP → `xTaskCreate()` for TCP task.

For details, review: **5. Network Protocols and Wireless Communication** -> **2. WIFI-STA Mode** -> **WIFI-STA Mode**

5.3 vTcpServerTask() — TCP Server Core

The TCP server has three extra steps compared to the client: `bind()` to occupy the port, `listen()` to start queuing, `accept()` to "answer the phone". It uses a **dual-loop structure**—the outer loop continuously `accept()` waits for the next client, the inner loop handles send/receive for the current client. When the client leaves, `close(client_fd)` returns to the outer loop to continue waiting.

`listen(fd, 5)`'s 5 doesn't mean "max 5 clients", but **max 5 clients queued waiting for accept**. Already-accepted clients don't count. Non-blocking `MSG_DONTWAIT` mode is also used for receiving data—if no data, skip; if data received, automatically reply.

```
void vTcpServerTask(void *pv)
{
    int listen_fd;           // Listen socket
    int client_fd;           // Client socket
```

```

struct sockaddr_in server_addr, client_addr;
socklen_t client_len;
char recv_buf[128];
char reply_msg[] = "Hello from ESP32 TCP Server!";
// Create TCP socket
listen_fd = socket(AF_INET, SOCK_STREAM, 0);

server_addr.sin_family = AF_INET;

// Listen on all IP
server_addr.sin_addr.s_addr = INADDR_ANY;
server_addr.sin_port = htons(TCP_SERVER_PORT);
// Bind port
bind(listen_fd, (struct sockaddr *)&server_addr, sizeof(server_addr));
// Start listening
listen(listen_fd, 5);          // Max 5 clients queued
ESP_LOGI(TAG, "TCP Server listening on port %d", TCP_SERVER_PORT);

while (1) {
    // wait for client connection (blocking)
    client_len = sizeof(client_addr);
    client_fd = accept(listen_fd, (struct sockaddr *)&client_addr, &client_len);

    if (client_fd < 0) {
        ESP_LOGE(TAG, "Accept failed");
        continue;
    }
    ESP_LOGI(TAG, "Client connected!");
    // Inner loop: communicate with current client
    while (1) {
        memset(recv_buf, 0, sizeof(recv_buf));
        int recv_len = recv(client_fd, recv_buf, sizeof(recv_buf)-1, MSG_DONTWAIT);

        if (recv_len > 0) {
            ESP_LOGI(TAG, "Receive from client: %s", recv_buf);
            send(client_fd, reply_msg, strlen(reply_msg), 0);
            ESP_LOGI(TAG, "Reply to client: %s", reply_msg);
        } else if (recv_len == 0) {
            // Client disconnected
            ESP_LOGI(TAG, "Client disconnected");
            break;
        }
        vTaskDelay(10);
    }
    close(client_fd);
}
close(listen_fd);
vTaskDelete(NULL);
}

```

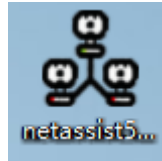
6. Operation Flow

6.1 Build and Flash Flow

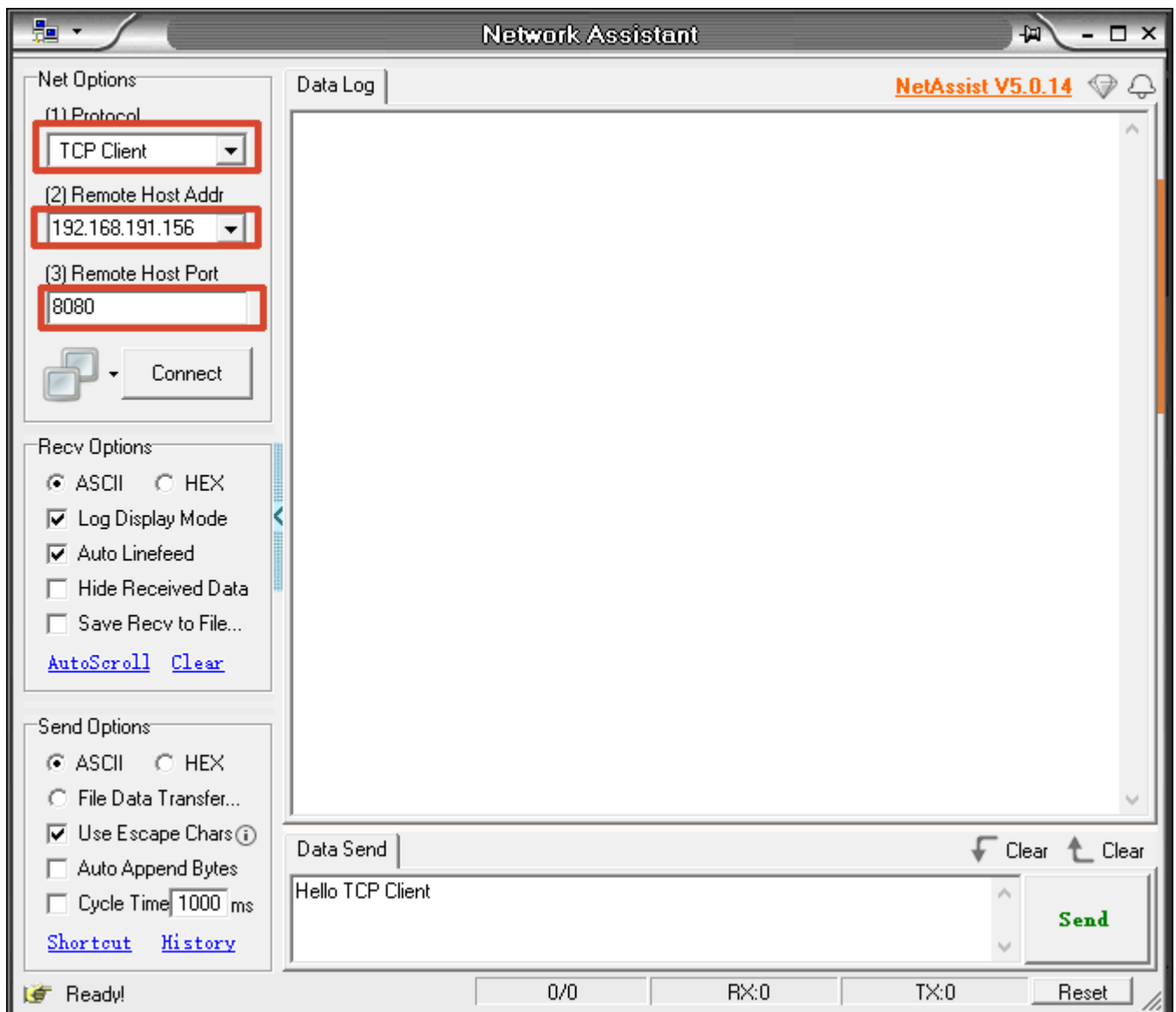
ESP-IDF Flash Flow: See `ESP-IDF` → 1. Before Development → 3. Build and Flash.

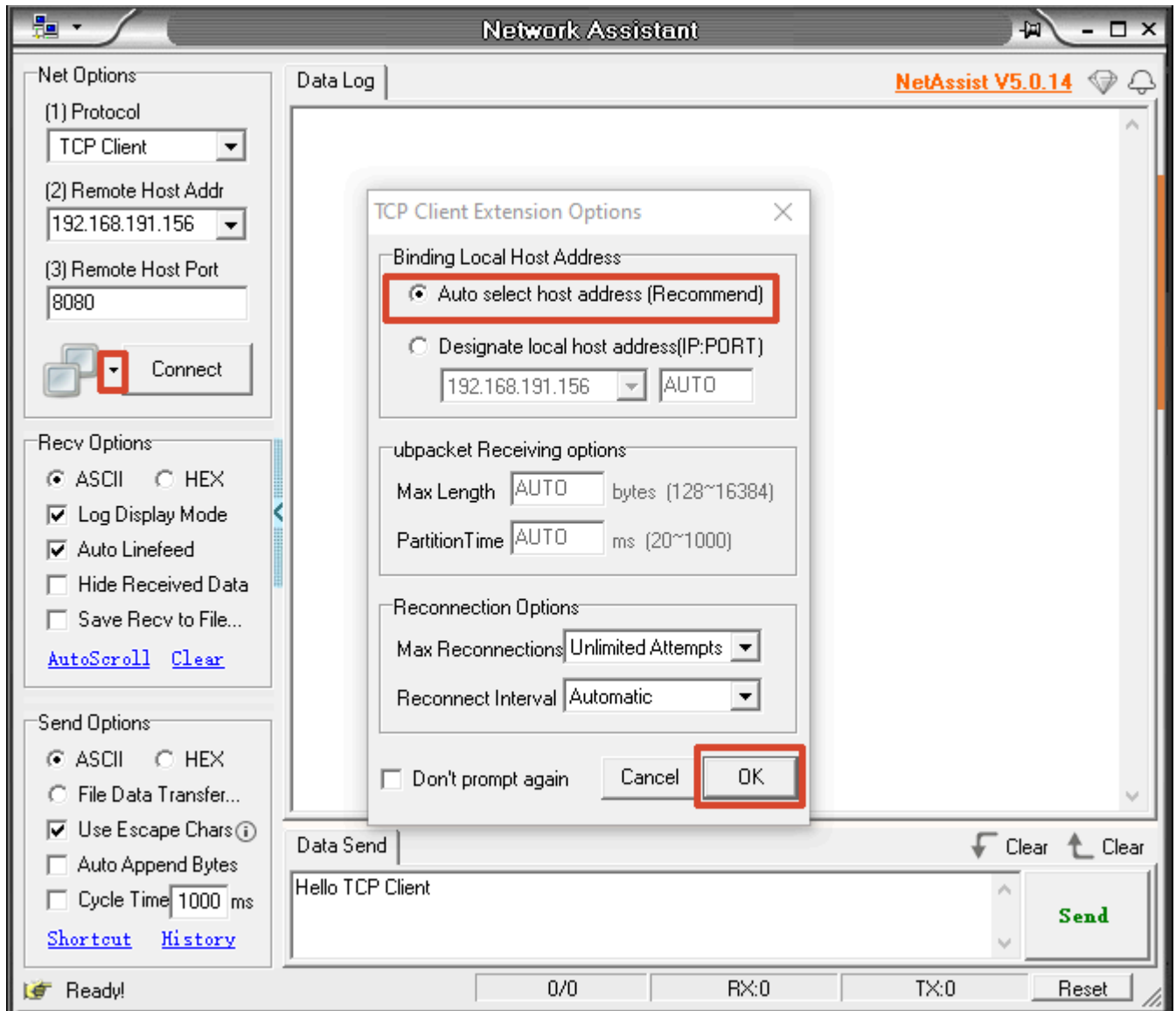
6.2 Testing Method

1. Run a network debugging assistant on PC, select TCP Server mode, listen on port `8080`.



2. **Flash TCP_Server** to ESP32. PC as TCP Client connects to ESP32's IP:8080, sends any data, ESP32 automatically replies.





```
[2026-07-11 19:07:23.306]# SEND ASCII/16 >>>
```

```
tello TCP Server
```

```
[2026-07-11 19:07:23.481]# RECV ASCII/28 from 192.168.11.181 :8080 <<<
```

```
tello from ESP32 TCP Server!
```

```
I (36754) TCP-SERVER: Reply to client: Hello from ESP32 TCP Server!
```

```
I (93554) TCP-SERVER: Client disconnected
```

6.3 Normal Log — TCP Server

```
I (2856) TCP-SERVER: Got IP: 192.168.11.100
I (2858) TCP-SERVER: WiFi connected successfully!
I (2860) TCP-SERVER: TCP Server listening on port 8080
I (5000) TCP-SERVER: Client connected!
I (5002) TCP-SERVER: Receive from client: ESP32 TCP Client Data
I (5004) TCP-SERVER: Reply to client: Hello from ESP32 TCP Server!
```

```

I (1714) TCP-SERVER: Got IP: 192.168.11.181
I (1714) TCP-SERVER: WiFi connected successfully!
I (1714) TCP-SERVER: TCP Server listening on port 8080
I (1724) main_task: Returned from app_main()
I (22654) wifi:<ba-add>idx:1 (ifx:0, 80:8f:1d:20:9d:f7), tid:0, ssn
:1, winSize:64
I (22654) TCP-SERVER: Client connected!
I (36754) TCP-SERVER: Receive from client: Hello TCP Server
I (36754) TCP-SERVER: Reply to client: Hello from ESP32 TCP Server!
I (93554) TCP-SERVER: Client disconnected

```

7. Common Problems

Symptom	Cause	Solution
<code>connect()</code> returns -1, connection failed	Server not started or wrong IP	Confirm PC TCP Server is listening, IP and port are correct
Server <code>accept()</code> blocks and doesn't return	No client connected	Normal behavior, it's a blocking call
Client <code>recv()</code> returns 0	Server actively closed connection	Normal disconnect, <code>break</code> then <code>close()</code>
Consecutive <code>send()</code> failures	Peer closed / network interrupted	TCP connection broken, need to <code>connect()</code> again
<code>bind()</code> failed	Port in use	Use a different port / wait 2 minutes (TIME_WAIT timeout)
<code>recv()</code> data "sticks" together	TCP is byte stream, no message boundaries	Define frame header+length or delimiter protocol at application layer
Server can only connect one client	Single-threaded sequential processing	Change to <code>xTaskCreate</code> for each <code>accept()</code> for independent tasks
Meaning of <code>listen()</code> second parameter 5	Not max connections, it's the pending connection queue length	Already-accepted connections don't count